

OS ORANGE PROBLEM

SRN: PES1UG24CS515 NAME: Varnika Sadineni Section: I

PHASE1:

1A

```
varnika@varnika-VirtualBox: ~/PES1UG24CS515-pes-vcs
varnika@varnika-VirtualBox:~/PES1UG24CS515-pes-vcs$ nano object.c
varnika@varnika-VirtualBox:~/PES1UG24CS515-pes-vcs$ make test_objects
./test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
object.c: In function 'object_write':
object.c:147:47: warning: '/tmpXXXXXX' directive output may be truncated writing 10 bytes into a region of size between 1 and 512 [-Wformat-truncation=]
 147 |     snprintf(temp_path, sizeof(temp_path), "%s/tmpXXXXXX", dir_path);
      |
In file included from /usr/include/stdio.h:980,
      from pes.h:8,
      from object.c:11:
In function 'snprintf',
      inlined from 'object_write' at object.c:147:5:
/usr/include/x86_64-linux-gnu/bits/stdio2.h:54:10: note: '__builtin___snprintf_chk' output between 11 and 522 bytes into a destination of size 512
   54 |     return __builtin___snprintf_chk (__s, __n, __USE_FORTIFY_LEVEL - 1,
      |
   55 |                                     __glibc_objsize (__s), __fmt,
      |
   56 |                                     __va_arg_pack ());
      |
gcc -o test_objects test_objects.o object.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
varnika@varnika-VirtualBox:~/PES1UG24CS515-pes-vcs$
```

1B

```
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
varnika@varnika-VirtualBox:~/PES1UG24CS515-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
varnika@varnika-VirtualBox:~/PES1UG24CS515-pes-vcs$
```

PHASE2:

2A

```

varnika@varnika-VirtualBox:~/PES1UG24CS515-pes-vcs$ make test_tree
./test_tree
Makefile:48: warning: overriding recipe for target 'test_tree'
Makefile:22: warning: ignoring old recipe for target 'test_tree'
gcc -o test_tree test_tree.o object.o tree.o index.o -lcrypto
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.

```

2B

```

All Phase 1 tests passed.
varnika@varnika-VirtualBox:~/PES1UG24CS515-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
varnika@varnika-VirtualBox:~/PES1UG24CS515-pes-vcs$ xxd .pes/objects/XX/YYY... | head -20
xxd: .pes/objects/XX/YYY...: No such file or directory
varnika@varnika-VirtualBox:~/PES1UG24CS515-pes-vcs$ xxd .pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a
1af8e11ec4f476 | head -20
00000000: 626c 6f62 2031 3800 4475 706c 6963 6174  blob 18.Duplicat
00000010: 6520 636f 6e74 656e 740a                e content.
varnika@varnika-VirtualBox:~/PES1UG24CS515-pes-vcs$ xxd .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9d
b5e5c4b9aafc12 | head -20
00000000: 626c 6f62 2031 3600 4865 6c6c 6f2c 2050  blob 16.Hello, P
00000010: 4553 2d56 4353 210a                ES-VCS!.
varnika@varnika-VirtualBox:~/PES1UG24CS515-pes-vcs$ xxd .pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e65
0eda471e566fcf | head -20
00000000: 626c 6f62 2031 3500 5465 7374 2069 6e74  blob 15.Test int
00000010: 6567 7269 5879 0a                egriXy.
varnika@varnika-VirtualBox:~/PES1UG24CS515-pes-vcs$

```

PHASE3:

3A

```

varnika@varnika-VirtualBox:~/PES1UG24CS515-pes-vcs$ make pes
./pes init
Makefile:48: warning: overriding recipe for target 'test_tree'
Makefile:22: warning: ignoring old recipe for target 'test_tree'
make: 'pes' is up to date.
Initialized empty PES repository in .pes/
varnika@varnika-VirtualBox:~/PES1UG24CS515-pes-vcs$ echo "hello" > file1.txt
varnika@varnika-VirtualBox:~/PES1UG24CS515-pes-vcs$ echo "world" > file2.txt
varnika@varnika-VirtualBox:~/PES1UG24CS515-pes-vcs$ ./pes add file1.txt file2.txt
varnika@varnika-VirtualBox:~/PES1UG24CS515-pes-vcs$ ./pes status
Staged changes:
  staged:   file1.txt
  staged:   file2.txt

Unstaged changes:
(nothing to show)

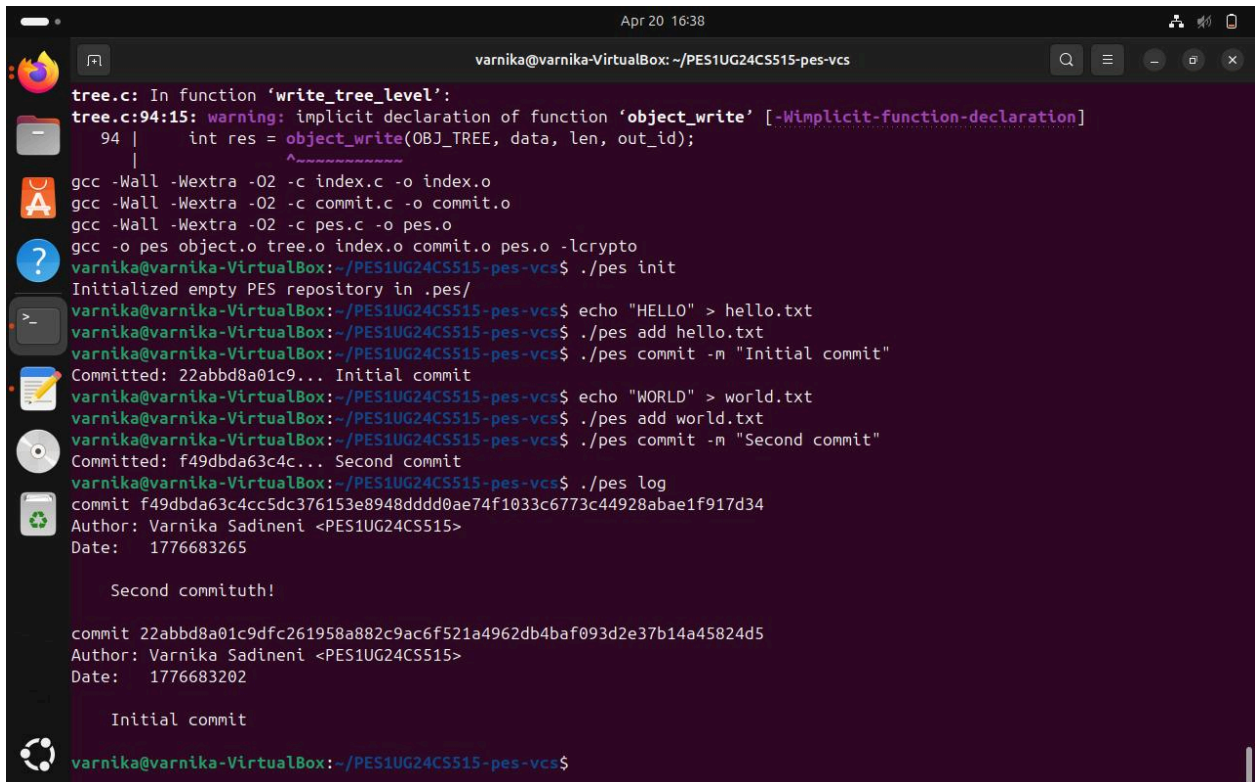
```

3B

```
varnika@varnika-VirtualBox: ~/PES1UG24CS515-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4 1776682622 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776682626 6 file2.txt
varnika@varnika-VirtualBox: ~/PES1UG24CS515-pes-vcs$
```

PHASE 4:

4A



```
varnika@varnika-VirtualBox: ~/PES1UG24CS515-pes-vcs$ gcc -Wall -Wextra -O2 -c index.c -o index.o
varnika@varnika-VirtualBox: ~/PES1UG24CS515-pes-vcs$ gcc -Wall -Wextra -O2 -c commit.c -o commit.o
varnika@varnika-VirtualBox: ~/PES1UG24CS515-pes-vcs$ gcc -Wall -Wextra -O2 -c pes.c -o pes.o
varnika@varnika-VirtualBox: ~/PES1UG24CS515-pes-vcs$ gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
varnika@varnika-VirtualBox: ~/PES1UG24CS515-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
varnika@varnika-VirtualBox: ~/PES1UG24CS515-pes-vcs$ echo "HELLO" > hello.txt
varnika@varnika-VirtualBox: ~/PES1UG24CS515-pes-vcs$ ./pes add hello.txt
varnika@varnika-VirtualBox: ~/PES1UG24CS515-pes-vcs$ ./pes commit -m "Initial commit"
Committed: 22abbd8a01c9... Initial commit
varnika@varnika-VirtualBox: ~/PES1UG24CS515-pes-vcs$ echo "WORLD" > world.txt
varnika@varnika-VirtualBox: ~/PES1UG24CS515-pes-vcs$ ./pes add world.txt
varnika@varnika-VirtualBox: ~/PES1UG24CS515-pes-vcs$ ./pes commit -m "Second commit"
Committed: f49dbda63c4c... Second commit
varnika@varnika-VirtualBox: ~/PES1UG24CS515-pes-vcs$ ./pes log
commit f49dbda63c4cc5dc376153e8948dd0ae74f1033c6773c44928abae1f917d34
Author: Varnika Sadineni <PES1UG24CS515>
Date: 1776683265

Second commituth!

commit 22abbd8a01c9dfc261958a882c9ac6f521a4962db4baf093d2e37b14a45824d5
Author: Varnika Sadineni <PES1UG24CS515>
Date: 1776683202

Initial commit
varnika@varnika-VirtualBox: ~/PES1UG24CS515-pes-vcs$
```

4B and C


```

varnika@varnika-VirtualBox:~/PES1UG24CS515-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/22/abbd8a01c9dfc261958a882c9ac6f521a4962db4baf093d2e37b14a45824d5
.pes/objects/35/f95ece271810bc0dbd32b8faa2d11855d6414bcd7c870aaf9e78f447d0c2e9
.pes/objects/69/de5b330550a563714c8d670747ed43b3c94f5bd54b09e963769cd125ecc152
.pes/objects/82/ef4cebb7afc20a47ffda706a42181db8f873090649fbcc857b30adba6ff1ff
.pes/objects/f4/9dbda63c4cc5dc376153e8948dddd0ae74f1033c6773c44928abae1f917d34
.pes/objects/fd/7356aec0491e94d9495b2a94742cecadd7d264e262ed63b1c94ddb51566393
.pes/refs/heads/main
varnika@varnika-VirtualBox:~/PES1UG24CS515-pes-vcs$ cat .pes/refs/heads/main
cat .pes/HEAD
f49dbda63c4cc5dc376153e8948dddd0ae74f1033c6773c44928abae1f917d34
ref: refs/heads/main

```

FINAL TEST

```

Apr 20 16:40
varnika@varnika-VirtualBox: ~/PES1UG24CS515-pes-vcs
git commit -m "Phase 4: Implement commits"
[main f86b8e9] Phase 4: Implement commits
1 file changed, 62 insertions(+), 5 deletions(-)
varnika@varnika-VirtualBox:~/PES1UG24CS515-pes-vcs$ make test-integration
Makefile:48: warning: overriding recipe for target 'test_tree'
Makefile:22: warning: ignoring old recipe for target 'test_tree'
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:    file.txt
  staged:    hello.txt

Unstaged changes:
(nothing to show)

Untracked files:
(nothing to show)

--- First Commit ---
Committed: 8a4347372fc3... Initial commit
Log after first commit:

```

```
varnika@varnika-VirtualBox: ~/PES1UG24CS515-pes-vcs
Log after first commit:
commit 8a4347372fc30959181b9017203abcc61ce687d29e81ea199cb0c1e1b6b79fc9
Author: Varnika Sadineni <PES1UG24CS515>
Date: 1776683423

Initial commit

--- Second Commit ---
Committed: 379bc625b1ab... Update file.txt

--- Third Commit ---
Committed: 119ace7d92e3... Add farewell

--- Full History ---
commit 119ace7d92e3ed42037952597902b16fce47727fede4ab58966c2819c4e8b2dc
Author: Varnika Sadineni <PES1UG24CS515>
Date: 1776683423

Add farewellauth!

commit 379bc625b1aba847f92622056dfce14c8f4686f7c1e9a8d9c15d0fb2ca40f381
Author: Varnika Sadineni <PES1UG24CS515>
Date: 1776683423

Update file.txth!

commit 8a4347372fc30959181b9017203abcc61ce687d29e81ea199cb0c1e1b6b79fc9
Author: Varnika Sadineni <PES1UG24CS515>
Date: 1776683423

Initial committial commiA
```

```
varnika@varnika-VirtualBox: ~/PES1UG24CS515-pes-vcs

Update file.txth!

commit 8a4347372fc30959181b9017203abcc61ce687d29e81ea199cb0c1e1b6b79fc9
Author: Varnika Sadineni <PES1UG24CS515>
Date: 1776683423

Initial committial commiA

--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
119ace7d92e3ed42037952597902b16fce47727fede4ab58966c2819c4e8b2dc

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/11/9ace7d92e3ed42037952597902b16fce47727fede4ab58966c2819c4e8b2dc
.pes/objects/37/9bc625b1aba847f92622056dfce14c8f4686f7c1e9a8d9c15d0fb2ca40f381
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/8a/4347372fc30959181b9017203abcc61ce687d29e81ea199cb0c1e1b6b79fc9
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
varnika@varnika-VirtualBox: ~/PES1UG24CS515-pes-vcs$
```

PHASE 5

Q5.1: Implementing pes checkout <branch>

A branch in PES-VCS is represented as a file under `.pes/refs/heads/` containing a commit hash, and HEAD points to the current branch reference. Implementing `pes checkout <branch>` involves both metadata updates and working directory changes.

Steps involved:

1. Validate the branch:
 - Check if `.pes/refs/heads/<branch>` exists.
 - If it does not exist, return an error.
2. Update HEAD:
 - Modify `.pes/HEAD` to point to the new branch:

 `ref: refs/heads/<branch>`
3. Read target commit:
 - Read the commit hash stored in `.pes/refs/heads/<branch>`.
4. Reconstruct working directory:
 - Parse the commit object to get the root tree hash.
 - Recursively traverse the tree:
 - For each blob, write its contents to the corresponding file path.
 - For each subtree, create directories and continue traversal.
 - Remove files that exist in the current working directory but are not present in the target tree.
5. Update index:
 - Replace the current index with entries corresponding to the checked-out tree.

Why checkout is complex:

- It requires synchronizing three states: the object store, index, and working directory.
- Handling file deletions, overwrites, and directory structures correctly is non-trivial.
- Conflicts may arise if uncommitted changes exist in the working directory.
- Ensuring atomicity and consistency during updates is essential to avoid repository corruption.

Q5.2: Detecting Dirty Working Directory Conflicts

When switching branches, the system must ensure that uncommitted changes are not overwritten.

Detection strategy:

1. Compare working directory with index:
 - For each file in the index:

- Check file metadata (size, modification time).
 - If metadata differs, recompute the file's hash and compare with the stored hash in the index.
 - If hashes differ, the file is modified (unstaged changes).
- 2. Compare index with target branch tree:
 - Identify files that differ between the current index and the target commit's tree.
- 3. Conflict condition:
 - If a file:
 - Has unstaged changes (working directory $\neq$ index), and
 - Differs between the current branch and target branch,
 - Then a conflict exists.
- 4. Action:
 - Abort checkout and notify the user to commit, stash, or discard changes.

Rationale:

This ensures that no local modifications are lost during checkout. The detection relies only on the index and object store, without needing a full diff algorithm.

Q5.3: Detached HEAD State

A detached HEAD occurs when `.pes/HEAD` contains a commit hash directly instead of a reference to a branch.

Behavior:

- New commits can still be created.
- Each new commit points to the previous commit via the parent field.
- However, no branch reference is updated to point to these new commits.

Consequences:

- The commits become unreachable from any branch.
- They may eventually be removed by garbage collection.

Recovery methods:

1. Create a new branch:
 - Create a file under `.pes/refs/heads/` containing the current commit hash.
 - Update `.pes/HEAD` to point to this branch.
2. Manual recovery:
 - Use the commit hash (if known) to create a reference.

Phase 6: Garbage Collection and Space Reclamation

Q6.1: Algorithm to Identify and Delete Unreachable Objects

Over time, objects may become unreachable if no branch or reference points to them. Garbage collection removes such objects.

Algorithm:

1. Identify root references:
 - Collect all commit hashes from:
 - `.pes/refs/heads/*`
 - Any other references (if present)
2. Traverse reachable objects:
 - Use a graph traversal (DFS or BFS):
 - Start from each root commit.
 - For each commit:
 - Mark it as reachable.
 - Add its parent commit (if any) to traversal.
 - Add its associated tree.
 - For each tree:
 - Mark it as reachable.
 - Traverse all entries:
 - If entry is a blob, mark it.
 - If entry is a subtree, recurse.
3. Track reachable objects:
 - Use a hash set to store visited object hashes for efficient lookup.
4. Delete unreachable objects:
 - Iterate over all files in `.pes/objects/`.
 - If an object's hash is not in the reachable set, delete it.

Data structure:

- A hash set (e.g., hash table) is used to store reachable object hashes efficiently, providing $O(1)$ lookup.

Complexity estimate:

- For 100,000 commits and 50 branches:
 - Many commits are shared across branches.
 - The traversal visits each unique commit, tree, and blob once.
 - Total objects visited is approximately:

- 100,000 commits
- Corresponding trees (similar order)
- Blobs depending on repository size
- Overall complexity is linear in the number of reachable objects.

Q6.2: Risks of Concurrent Garbage Collection

Running garbage collection concurrently with commit operations can lead to race conditions.

Race condition scenario:

1. A commit operation:
 - Writes new blob and tree objects.
 - Has not yet updated the branch reference.
2. Simultaneously, garbage collection runs:
 - It scans references and identifies reachable objects.
 - Since the new commit is not yet referenced, its objects appear unreachable.
3. Garbage collector deletes these objects.
4. The commit process then updates the reference to point to a commit whose objects no longer exist.

Result:

- Repository corruption due to missing objects.

How real systems avoid this:

1. Object creation before reference update:
 - Objects are written fully before updating references.
2. Atomic reference updates:
 - Reference files are updated atomically to ensure consistency.
3. Grace period (reflogs):
 - Recently created or modified references are retained temporarily.
4. Locking mechanisms:
 - Prevent garbage collection from running during critical operations.
5. Mark-and-sweep safety:
 - Only delete objects that have been unreachable for a certain duration.